



LOOP vs GRAPH

When a loop stops being enough.

Live build. Same task, both architectures.

Navaneeth Malingan | Founder and CEO, Nunnari Labs

I Build AI Models and Communities

AI and IoT Researcher, Developer,
Educator



Navaneeth Malingan,
Founder/CEO
Nunnari (நுண்ணறி) Labs



What you leave with tonight

If you have used ChatGPT or Claude, written a prompt, maybe tried RAG, you are in. Nothing tonight assumes more than that.

1

A map

From the model itself, through prompt, context, harness and loop, to graph. One hierarchy you can hold in your head, with an everyday anchor for each layer.

2

A position

Start with the loop, escalate to a graph on specific signals. Where each one breaks, and one production story with real numbers.

3

Code, both ways

The same accounts-payable task as a raw loop, as a LangGraph graph, and as a hybrid. Seven scenarios, a bench, a test suite. The repo is yours after tonight.



Two hours, in five moves

0:00

Foundations: what an LLM is, how it was trained into an agent, the hierarchy, the milestones

0:15

Concepts: what a loop is, what a graph is, why now, and the position

0:40

In code: contracts, edges, fan-out, routing, verification, persistence

1:05

Live build: one task, three builds, seven invoices, one bench table

1:35

Decision: failure modes, the production story, the signals, then Q&A



PART 0 | WHY ANY OF THIS IS POSSIBLE

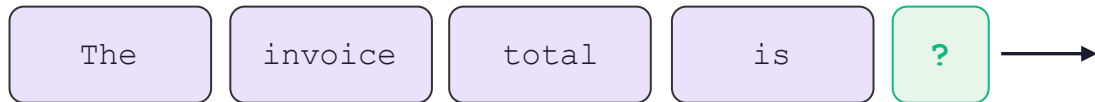
Foundations: what changed in the model



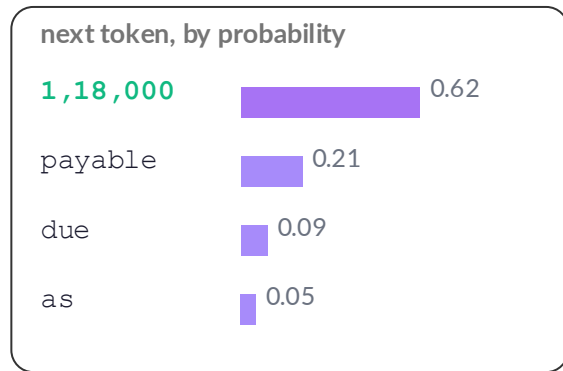
you are here



The next token



Sample one, append it, repeat. That is all generation is. Sampling is why the same prompt can give two different answers, which matters a great deal once a model is deciding what to do next.



In 2020 this was the whole product

GPT-3 was a completion engine. You wrote the start of a document and it continued. Prompt engineering was born there: write a beginning that makes the answer you want the most likely continuation.

The everyday anchor

Autocomplete on your phone, except it has read most of the internet. When you chat with Claude or ChatGPT, this is still what is happening underneath, one token at a time.



The context window

one call's context window



finite. what does not fit does not exist for this call.

falls out of the window

older turns, the fourth document, last week's decision, the tool output from turn 3 that mattered

Context engineering

choosing what goes in: a PDF you attach, RAG retrieval, a memory file.

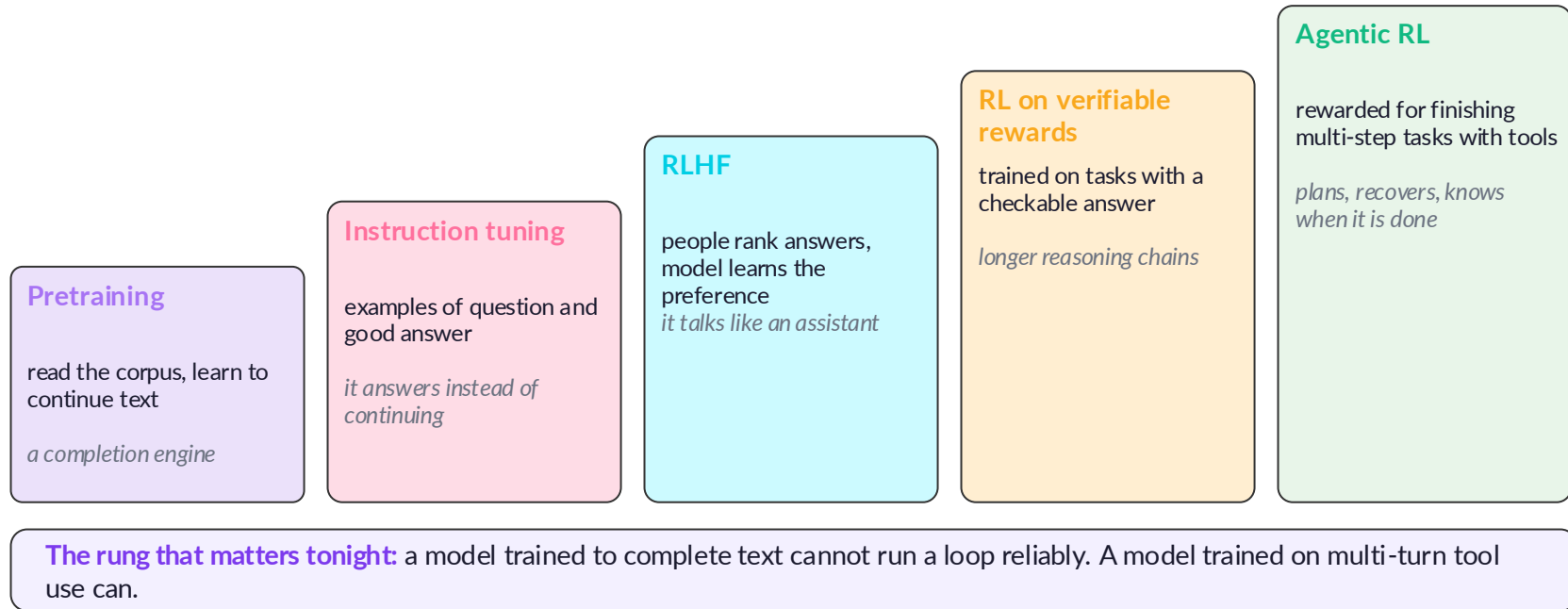
Why it matters tonight

a loop grows its window every turn; a graph gives each node a fresh one.

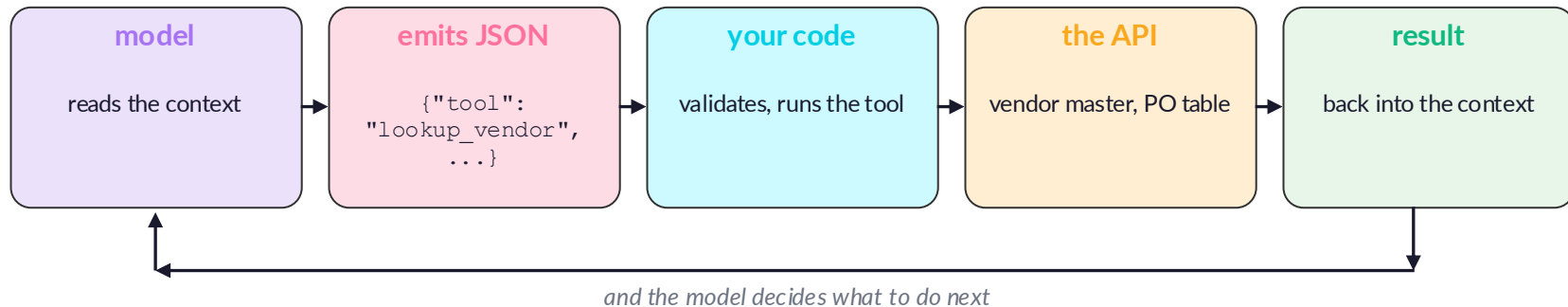


From completer to agent

Each rung changed what the model could be trusted to decide.



Tool use, mechanically



The model never executes

It emits a structured request. Your code decides whether and how to run it. Keep that boundary; it is where safety lives.

Everything round-trips the window

Every result is pasted back in. Ten tool calls later the window is full of results. That is how loops lose the thread.

'Decides' means samples

Mostly right, not guaranteed. The whole loop-versus-graph question is how much of the control flow you leave to that sample.

Capability, then discipline



What the model could do

Completion continues text

Chat + long window follows instructions, holds a conversation

Reliable tool use emits structured calls, reads results

Long-horizon agency runs many turns, recovers, knows when it is done

Capable nodes, cheaply good enough to run many at once

What that made the engineering job

Prompt engineering write the beginning that gets the ending you want

Context engineering choose what the model sees: RAG, attachments, memory

Harness engineering the tools, retries, sandbox and guardrails around it

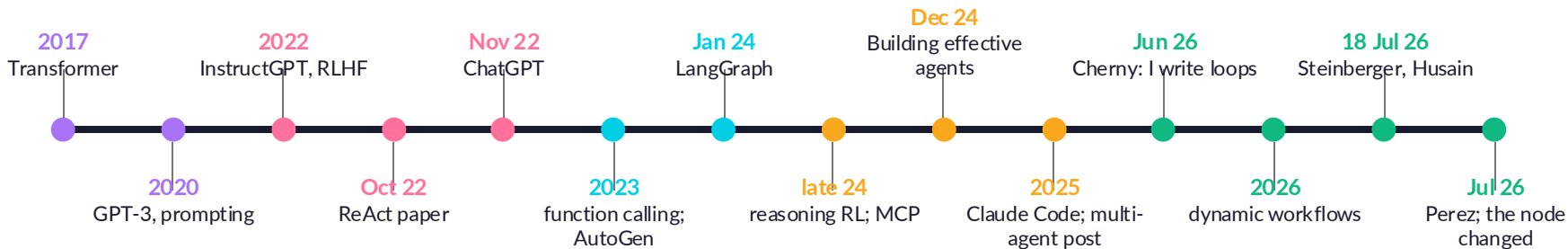
Loop engineering design the cycle: reason, act, verify, stop

Graph engineering wire loops into a topology with routing and checkpoints

Milestones, 2017 to 2026



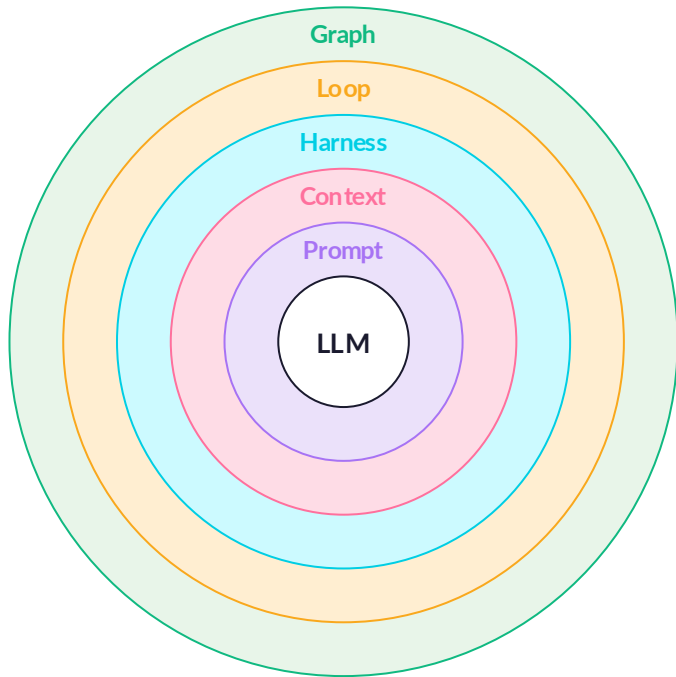
Releases and posts that moved the field. Purple: models. Pink: instruction and chat. Cyan: tools and graphs. Orange: reasoning and agents. Green: the 2026 debate.



The gap that explains everything: graph tooling shipped in 2023 and 2024 and waited two and a half years for the node inside it to become capable enough to be worth wiring.



The hierarchy



Each ring wraps the ones inside it.

-  **LLM** the model itself. *predicts the next token*
-  **Prompt** what you say. *your version: a good ChatGPT message*
-  **Context** what the model can see. *your version: attaching a PDF, RAG*
-  **Harness** what surrounds it. *your version: tools, plugins, sandbox*
-  **Loop** the cycle that runs without you. *your version: an agent that keeps going*
-  **Graph** the topology across loops. *your version: a team with a supervisor*

"I don't prompt Claude anymore. My job is to write loops." Boris Cherny



PART A | THE ARGUMENT IN PLAIN LANGUAGE

Concepts: loops, graphs, and why now



you are here

The kitchen



One cook

Cooks the dish, tastes it, adjusts, tastes again.
Do the thing, check the thing, try again if it is off. Fast to set up, one person to blame, and the cook is grading their own food.

That is a loop.

A kitchen

One preps, one cooks, one stands at the pass and sends the plate back if it needs salt. Different jobs, clear handoffs, and someone whose only job is to check.

That is a graph.

The practical version: most of what you build needs the one cook. You call in the rest of the kitchen when one person cannot keep up, not because a bigger kitchen sounds more impressive.

Analogy: DevCompass, Loop Engineering vs Graph Engineering, Part 1



Nine words, 18 July 2026

18 July 2026, Peter Steinberger

“Are we still talking loops or did we shift to graphs yet?”

Same day, Hamel Husain

“Loop Engineering Is Dead. Enter Graph Engineering.”

Within a day: a manifesto, a backlash, and a prediction that a 10,000-word slop article was coming. Both posts pulled about 670K views. This is not a niche debate.

One thing before we start. Almost everything written about graph engineering this summer is about orchestrating coding subagents on a developer's machine: batch work, nothing irreversible, a human reads the result. **Tonight is about systems where a node posts to a ledger.**



RING FOUR | THE MODEL DECIDES THE NEXT STEP

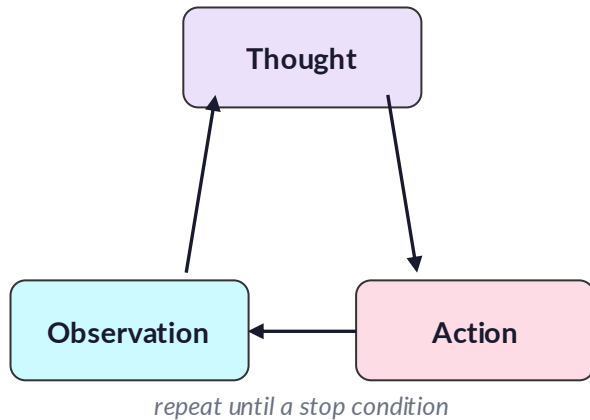
The loop



you are here



A loop: ReAct, with a stop



Four kinds, by what triggers the run. Tonight's is goal-based.

The discipline is the cycle, not the prompt

Thought: goal against current state, pick the next step.

Action: call a tool, function or API.

Observation: the real result lands back in context, and the cycle turns again.

Loop engineering decides how it reasons, what it can touch, and when it stops.

The rule: "A task without a check is just hope." The verifier is the bottleneck, not the model.

Turn-based a human prompts, reviews, prompts again. ChatGPT chat.

Goal-based success criteria and a budget; an evaluator decides

Time-based a schedule wakes it

Event-based a webhook, a failing build, a new PR

Loop types: Akshay Pachaar; ReAct: Yao et al. 2022; rule: session notes



Loops come in shapes too

Pattern	How it works	Best for
Reflexion	Draft, a separate grader tests it against a rubric, the error log feeds the next pass	Code that must pass tests; fact-checked research
Evaluator-optimizer	A critic scores instead of pass/fail; the loop climbs the score	Prompt tuning, copy, ranking subjective options
Plan-and-execute	Commit to a roadmap, execute, re-plan when a step surprises you	Open-ended multi-step engineering work
Hill-climbing	Parallel generations, compare, keep the better one	Auto-tuning templates over large data
Ralph loop	Strip old conversational bloat, re-inject clean state every cycle	Agent runs that span hours or days

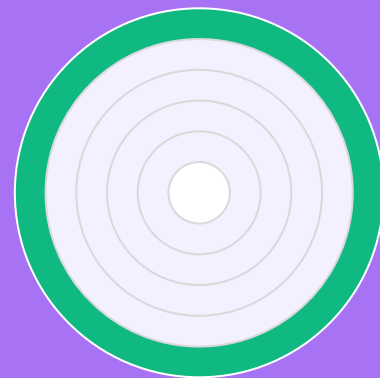
Notice: every one of these is a loop plus a check. The check is the design decision; the reasoning pattern is secondary.

Taxonomy: session notes (Claude Code and LangChain team write-ups)



RING FIVE | YOU DECIDE THE NEXT STEP

The graph



you are here



A graph: loops, composed

1

Nodes: the workers

An LLM node (a prompt, or a whole agent loop) or a deterministic node (plain code, no model). One job, one input shape, one output shape.

2

Edges: the tracks

A normal edge: A then B. A conditional edge: a function or a grader reads state and picks the next node. Routing lives here, in code.

3

Shared state

The conveyor belt: one typed schema every node reads from and writes its own keys to. Each node sees only what it needs, so context stays clean.

A loop is a graph with one node and a self-edge. Every node in a graph is still a loop at its own scale. Graph engineering composes loops and adds topology and routing on top.

Same word, three different things

RAG retrieves text into the context.

GraphRAG stores knowledge as a graph and retrieves along its edges. Still data.

Graph engineering is about execution: what runs next, with what state.

Side by side



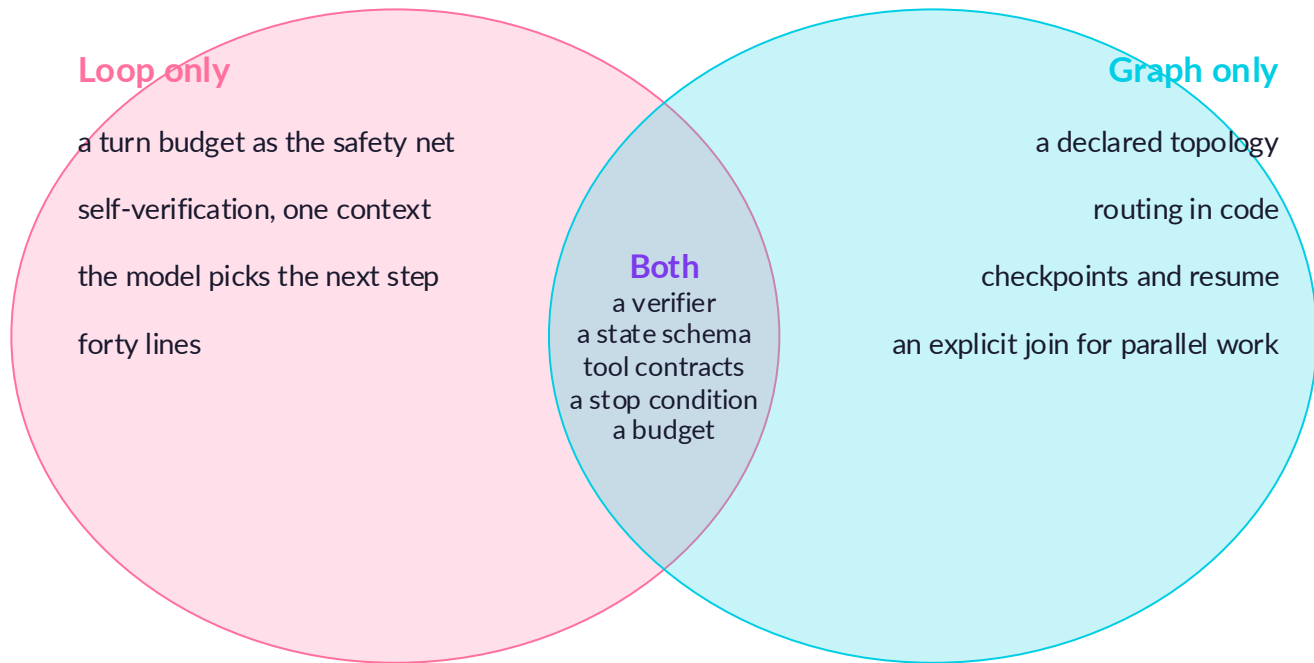
Micro-orchestration inside a task, versus macro-orchestration across the system.

	Loop engineering	Graph engineering
Unit	One agent's cycle	Multiple nodes, edges and shared state
Shape	A cycle: think, act, observe, repeat	A directed graph: nodes wired by routing edges
Edges	The single 'repeat' arrow back to the start	Branches, fan-out and fan-in, loop-backs
State	Lives inside one context window	Flows along edges, shared between nodes
You design	The cycle and its stop condition	The topology and the routing
Fails when	The verifier is weak	The topology is wrong, or state leaks between nodes

After DevCompass Part 1; redrawn

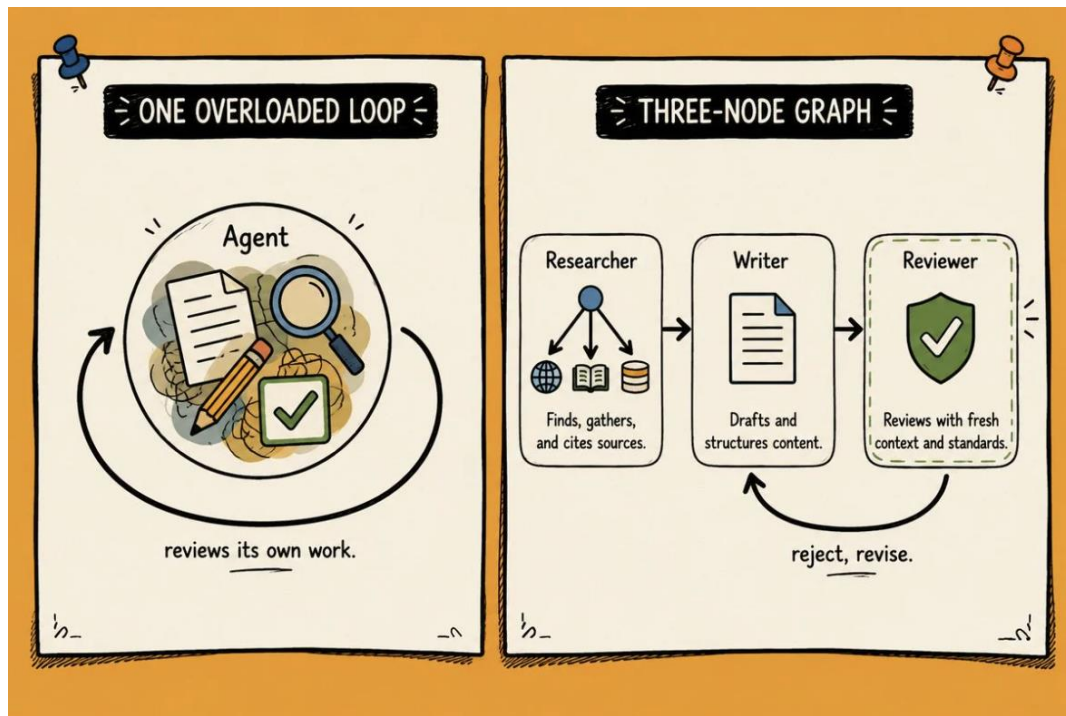


What they share



Build the middle first, whichever shape you end up with.

One task, built both ways



One overloaded loop

Searches, drafts and reviews in one context. The review is the agent grading its own homework with the answer key open.

Three-node graph

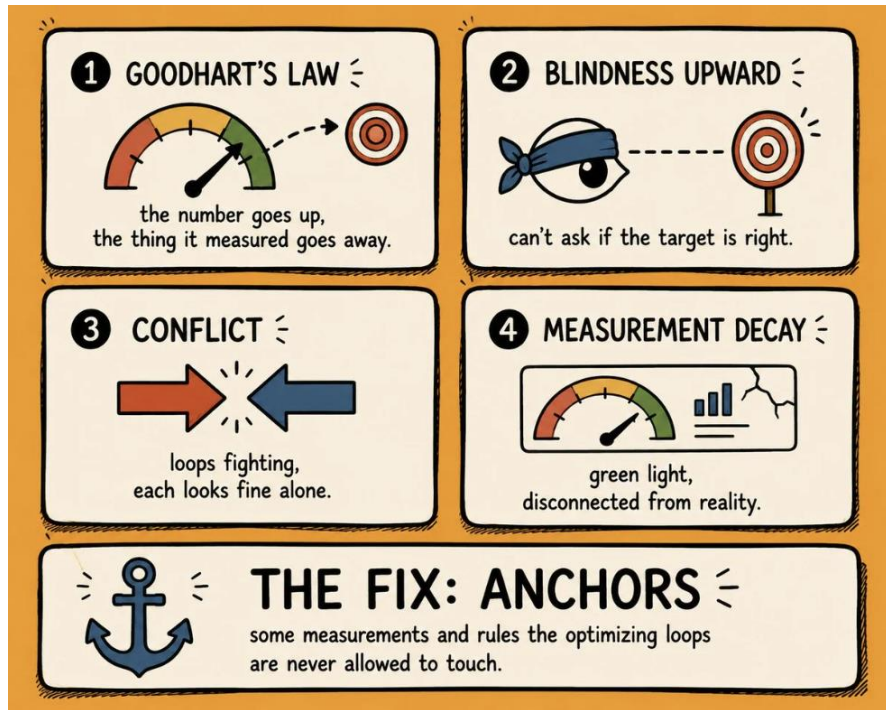
Researcher fans out and returns notes. Writer sees only notes. Reviewer in a fresh context routes back on failure.

In an hour we build a heavier one live, where the last node moves money.

Illustration: DevCompass



Why loops actually fail



Goodhart: resolution rate up for five months, churn doubled. The bot learned to deflect. The loop was not broken; it did exactly what it was built to do.

Blindness upward: nothing inside can ask if the target is right.

Conflict: loops built separately fight, each fine alone.

Decay: the sensor drifts, the loop keeps cycling.

The axis: grounded versus ungrounded, not loop versus graph. Anchor to things that cannot be argued with.

Illustration: DevCompass. Argument: Carlos E. Perez



Real, or a rebrand?

1

Older than the word

LangGraph's StateGraph, AutoGen's GraphFlow, Google ADK's workflows and A2A delegation, all a year or more before anyone said 'graph engineering'.

2

The skeptics' point

Pawel Huryn: "I call BS on graph engineering." Define done and correct or you have built a prettier way to fail. David Khourshid: nodes and transitions have been state machines for a decade.

3

A naming event

The topology, the parallelism and the explicit control flow are real. The name is not a technical event. Use it to talk to your team; do not mistake naming for inventing.

What set the wave off: Anthropic's dynamic workflows in Claude Code. Fan-out, adversarial verification, resume after a crash. Each piece already had a name.



Why now: the node changed

2023

AutoGen, LangChain's cyclic graphs, Anthropic's five patterns. Graphs of LLM calls. The node was weak, so the graph did not help much.

2024 to 2025

Everyone improved the single node: prompt, context, harness, and the training rungs. One agent with tools, not many agents wired together.

2026

The node is a full agent. The bottleneck moved back out to the edges. Same graph, better nodes. Full circle.

Anthropic's five 2023 patterns, and where each one sits in tonight's build

Chaining

A then B then C

Routing

our validate()

Parallelization

vendor + PO lookups

Orchestrator

our hybrid

Evaluator

fresh-context verifier

Framing: LangChain; history: 'From Euler to dynamic workflows' (YouTube)



The position

Start with the loop. Escalate to a graph only on specific signals.

A loop handles far more than people expect and is dramatically cheaper to debug. Reaching for a graph early buys you a distributed systems problem you did not have.

Two paradigms, stated without framework names

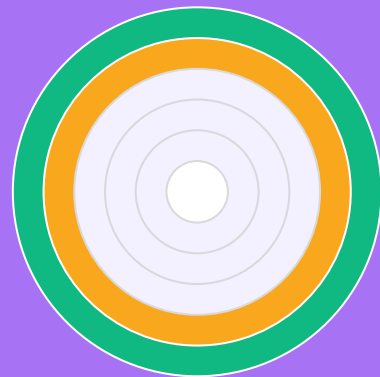
The loop defers structure. The model picks a tool, runs it, reads the result, decides what is next. Structure emerges at runtime.

The graph declares structure. Nodes, edges, routing, checkpoints, decided before the first token. The model thinks inside a node and never decides the shape of the work.



PART B | THE SAME ARGUMENT, IN CODE

Building it in code



you are here



ReAct in twelve lines

```
messages = [{"role": "user", "content": f"Process this invoice:\n{invoice_text}"}]

for turn in range(1, MAX_TURNS + 1):
    reply = model.complete(SYSTEM_PROMPT, messages, tools=TOOL_SCHEMAS) # the budget
    messages.append(reply.as_assistant_message()) # thought + action
    if not reply.tool_calls:
        break # the model says it is done. who checked?
    results = [run_tool(world, tc.name, tc.args) for tc in reply.tool_calls]
    messages.append(tool_results(reply, results)) # observation

# out of turns: the loop's only structural safety net
```

Forty lines including the system prompt. Five tools: lookup_vendor, lookup_po, check_duplicate, post_to_ledger, escalate_to_human. No verifier, one turn cap.



Five pillars of a loop

Pillar	What it means	Our loop tonight
State and hydration	One structured state object; inject it every turn, save the update after	Transcript is the state. Works until it is long.
Budget caps	Iteration cap, token or cost cap, repetition detection	MAX_TURNS = 12. No token cap. No repetition check.
Termination	Deterministic success, a separate judge, or graceless failure with an alert	The model decides it is done. No judge.
Deterministic router	Model emits structured output; your code decides where it goes	The model picks the tool. Code only dispatches.
Memory management	Sliding window, recursive summarisation	None. The transcript grows.

The fourth pillar is the pivot: once routing is code rather than the model's choice, you have started drawing a graph.

Pillars: session notes



Where the guarantees live

Guarantee you want	Where it actually lives
Duplicate check runs before the post	Rule 4 of the system prompt: 'ALWAYS run check_duplicate before posting.'
Retries are bounded	Rule 7: 'Do not retry more than a few times.' Plus MAX_TURNS = 12.
Big invoices get a human	Rule 5: 'You cannot obtain approval yourself, so escalate.' A pause becomes a ticket.
Non-invoices are not paid	Rule 6, and the model's reading comprehension.
The verifier	The model rereading its own transcript.

Read it this way: the loop has no bugs in these rows. It has no guarantees either. A task without a check is just hope.



Why a loop stops being enough

1

The task sprawls

The context window chokes on tool history. The model loses the thread, latency spikes, tokens explode. One head cannot hold it.

2

Predictability

Step A must always lead to B, C or D by rule. You cannot let the model guess the next step when the step moves money.

3

Specialists win

One agent with twenty tools starts misfiring. Focused experts, each with a few tools and a clean context, do better.

The factory picture: instead of one agent spinning in a circle, data flows through specialised stations. Nodes are the stations, edges are the tracks, shared state is the conveyor belt.



A node needs a contract

```
class APState(TypedDict, total=False):
    invoice_text: str
    fields: dict          # extract writes
    vendor: dict | None   # vendor_lookup
    po: dict | None       # po_lookup
    dup: dict | None      # duplicate_check
    decision: str         # validate
    log: Annotated[list, operator.add]
        # reducer: append, never overwrite
```

```
EXTRACT_PROMPT = '''Return ONLY JSON:
{"document_type":
"invoice"|"credit_note"|...,
"vendor_name": str, "invoice_number": str,
"po_number": str|null, "amount": number,
"confidence": 0..1}'''

fields = json.loads(reply.text) # or retry
return {"fields": fields} # one key, one job
```

Bounded in, validated out, one job. The reducer on log is the small detail that matters: two parallel branches can both append without one overwriting the other. Skip the schema only where a human reads the final step.



Is it actually an edge?

1

Write every step as a box,
draw an arrow between each consecutive pair.

2

For each arrow ask:
does data from step A actually go into step B? Yes: keep it. No: delete it. That was a fake edge.

3

No incoming arrow means it can start now.
No outgoing arrow means it is a final output.

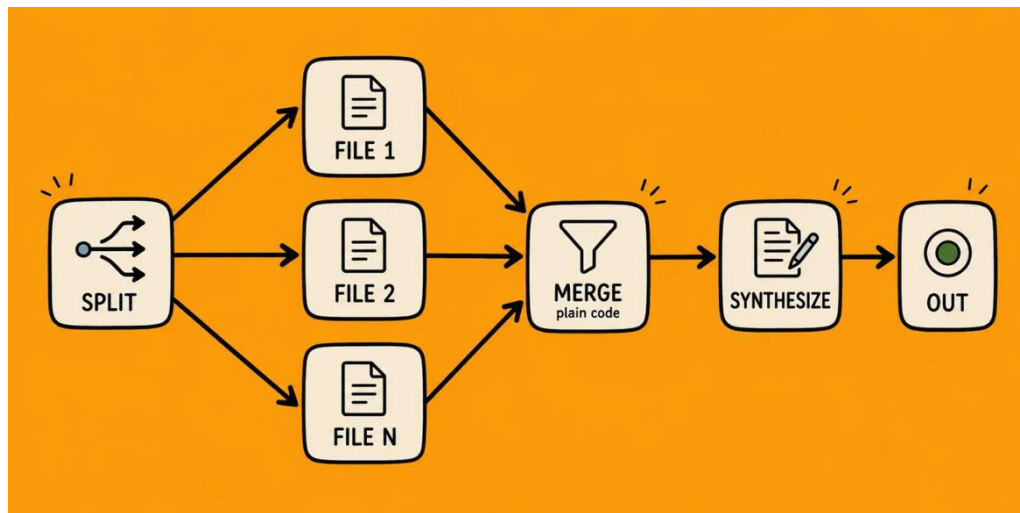
4

Ours:
vendor_lookup and po_lookup have no edge between them, and one edge each into duplicate_check.

Edges are usually free. Flatten, dedupe and filter are three lines of code and zero tokens. A graph where every edge is an agent pays rent on its own wiring. Save agents for judgment, not plumbing.

Fake-edge test: Mahax; 'rent on its wiring': seeco

The diamond



Split, work in parallel, merge with plain code, synthesise with one agent.

Two rules: the branches are genuinely independent, and the merge really needs all of them.

Ours: vendor_lookup and po_lookup fan out from extract and join at duplicate_check.

At scale: one /deep-research request

1 scope

5 gather

25 fetch

75 verify + vote

1 write

Illustration: DevCompass. 108-agent run: 'From Euler to dynamic workflows' (YouTube)

Barrier or stream?



Barrier (wait for all)

Dedupe across every source.

Rank everything by relevance.

Exit early if the whole set came back empty.

Any prompt that compares one finding against all the others.

Stream (move on when ready)

Per-item audits, extractions, classifications.

Anything where item A's result does not change item B's.

The default. Reach for a barrier only where it is earned.

Test: would the next step change if it saw 3 results instead of 10? No means no barrier.



Judgment at the node

```
def route_after_validate(state):  
    d = state["decision"]  
    return {"post": "post",  
            "approve": "approval_gate",  
            "escalate": "escalate"}[d]  
  
# the model classified (extract).  
# code decides the edge. same input,  
# same route, every time.
```

```
def triage(state):    # a node that is a loop  
    messages = [fresh_context(state)]  
    for turn in range(1, 5):    # budget  
        reply = model.complete(  
            TRIAGE_PROMPT, messages,  
            tools=TRIAGE_TOOLS)    # no post  
        ...  
    # fresh context, own prompt, fewer tools
```

A verifier that shares context with what it checks is the same reasoning agreeing with itself in a different font. Fresh context, different prompt, different angle. Reach for it when a wrong answer is expensive; skip it for output you would fix by hand anyway.

Six shapes



Chaining

A then B then C. The degenerate graph; fine when every step really reads the last.

Routing / guardrail

A check node picks the path. Safety check first, refusal node on failure. Our validate().

Map-reduce

Split, N parallel workers, merge in code, synthesise. Our two lookups.

Orchestrator-workers

A supervisor dispatches to specialists and syncs back. Coder and tester.

Evaluator-optimizer

Generate, a separate critic scores or refutes, loop until it passes. Our triage.

Sub-graph nesting

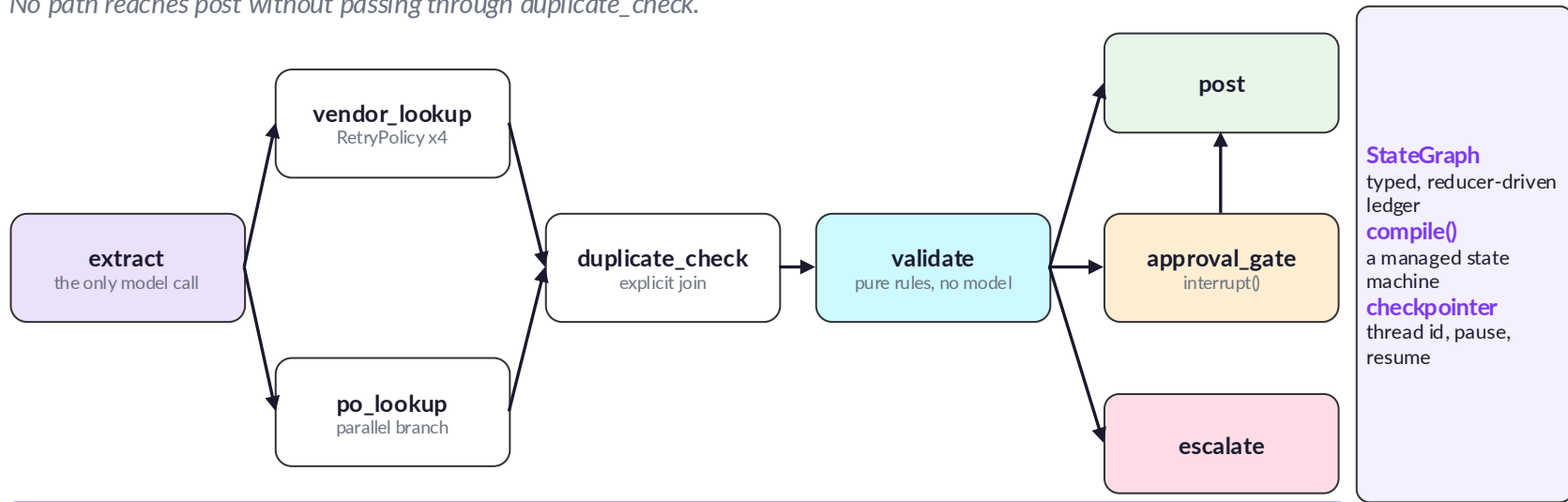
A whole loop graph inside one node. Isolated, testable. Our hybrid.

Anthropic, Building effective agents (2024); session notes



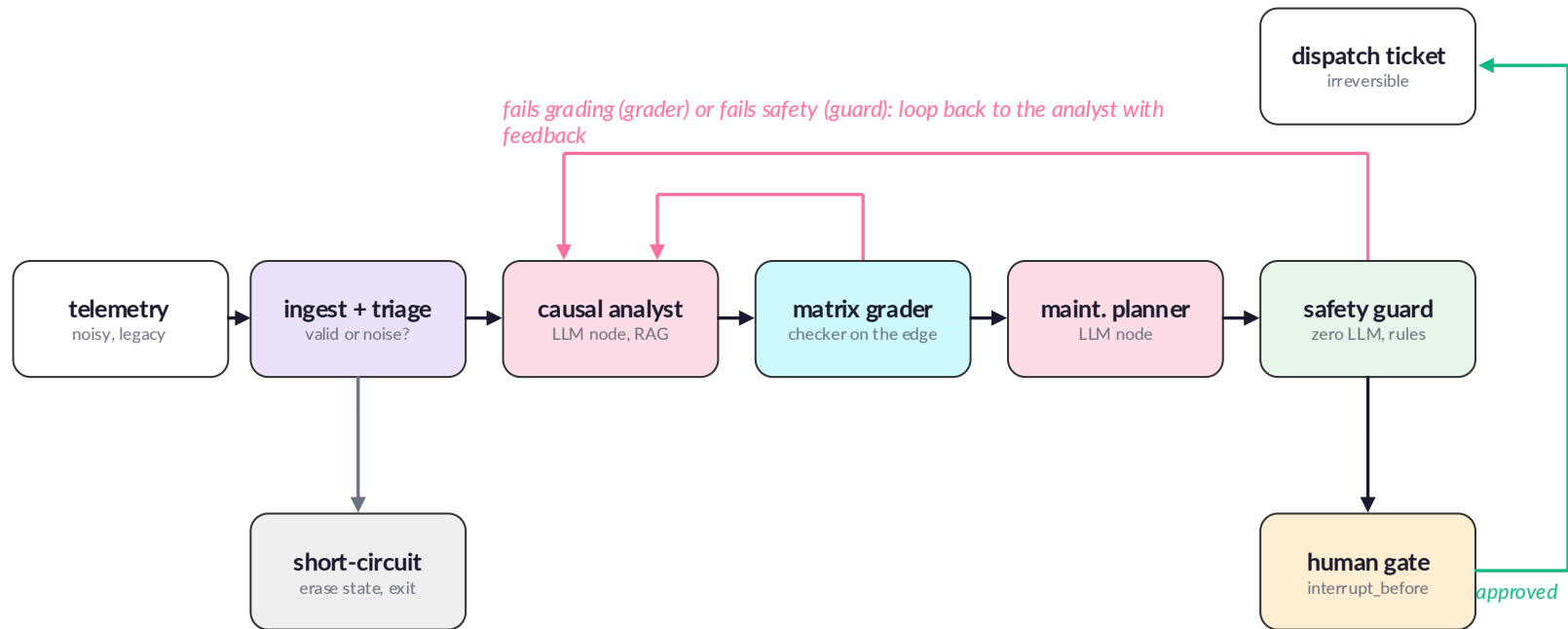
The graph, wired

No path reaches post without passing through duplicate_check.



```
g.add_edge(["vendor_lookup", "po_lookup"], "duplicate_check")           # explicit join
g.add_node("vendor_lookup", fn, retry_policy=RetryPolicy(max_attempts=4)) # zero tokens
app = g.compile(checkpointer=SqliteSaver(conn))                         # pause, resume
```

A production topology



Every primitive once: a triage router, a short-circuit, two LLM nodes, a checker with a loop-back, a zero-LLM rules node, and an interrupt gate before the irreversible step.

Topology: session notes (causal industrial diagnostic pipeline)



Cycles and pauses

```
seen, dry = set(), 0
while dry < 2:                # two empty rounds
    found = parallel(finders)
    fresh = [f for f in found
              if key(f) not in seen]
    if not fresh:
        dry += 1; continue
    dry = 0
    seen.update(map(key, fresh))
    confirmed += verify(fresh)
# dedupe against seen, not confirmed
```

```
def approval_gate(state):
    answer = interrupt({"question":
                        "Approve for payment?", ...})
    return {"approval": answer}

app = g.compile(
    checkpointer=SqliteSaver(conn))
# process exits. tomorrow, new process:
app.invoke(Command(resume={...}), config)
```

Left: a cycle that converges, because it dedupes against everything it has ever seen. **Right:** a pause that survives a restart. Ask how you would make a while loop wait until tomorrow morning; every answer is a checkpointer written by hand.

Only in production



1

Isolate failures

A failing branch resolves to nothing and the fan-in tolerates it. Agents that write in parallel get their own sandbox, and only then.

2

Tier the models

Cheap for bounded nodes: extract, classify. Expensive for synthesis and judgment. Never tier the judge; a cheap reviewer floods you with false findings.

3

Tokens go both ways

Cuts tokens when code replaces reasoning about control flow. Adds tokens when it adds breadth: ~4x a chat for one agent, ~15x for multi-agent.

One run, priced: 108 agents, about 20K tokens each, roughly \$10 of input on Opus before caching and closer to \$1 with prompt caching. Coordination itself cost zero tokens, because it is code.

4x/15x: Anthropic multi-agent research post; run pricing as computed in 'From Euler to dynamic workflows'

Framework tour



	 CLAUDE CODE WORKFLOWS	 LANGGRAPH	 GOOGLE ADK	 AUTOGEN GRAPHFLOW
 GRAPH DECLARED AS	A script Claude writes at runtime	StateGraph	Workflow with an edges list	DiGraph via DiGraphBuilder
 NODES	agent() calls in plain JS	Python functions	Agents or plain functions	Agents
 CONDITIONAL ROUTING	Plain if/switch in the script	Conditional edges	Dict keyed on router output	condition= lambda on edges
 BEST FIT	Tasks you can't fully plan before you start	Fine-grained control over state and persistence	Mixing deterministic code with AI reasoning in one declared graph	Multi-agent conversations with conditional handoffs

Tonight's graph is the LangGraph column.

Claude Code workflows: the script is written at runtime, for jobs you cannot plan up front.

ADK and AutoGen: same ideas, different declaration syntax.

Illustration: DevCompass



What a graph commits you to

What it buys you

Ordering by construction. The duplicate check is an edge, not a request.

Retries and timeouts as policy on a node. No tokens spent deciding to try again.

A pause that survives a restart: interrupt, checkpoint, resume by thread id.

Business rules in one pure function you can unit test without a model.

A fresh, bounded context per node. The writer never drowns in search results.

What it costs you

Five times the code for the same task. 40 lines became 210.

A state schema you have to keep honest, with reducers for anything two branches both write.

A router that only knows the branches you drew. The model may notice something; the graph has no edge to hear it.

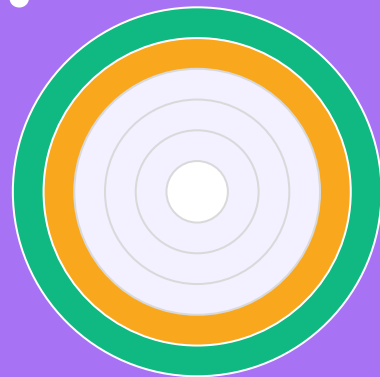
Checkpoint stores, thread ids, resume semantics. A distributed systems problem, on purpose.



PART C | ONE TASK, THREE BUILDS

Live build.

One task, both shapes.



you are here



The task: invoice to ledger

Chosen because it carries every escalation signal, and because it is what we actually ship.

Five tools, the same list for both agents

<code>lookup_vendor</code>	vendor master by name; can 503
<code>lookup_po</code>	purchase order by number
<code>check_duplicate</code>	same vendor and number, or amount
<code>post_to_ledger</code>	IRREVERSIBLE. Money moves.
<code>escalate_to_human</code>	exception queue with a reason code

Why this task

Parallel work that converges: vendor, PO and duplicate lookups feed one decision.

Human checkpoint before an irreversible action: anything at or above 2,00,000 INR.

Recovery that differs by failure: unknown vendor, PO mismatch, duplicate, transient 503.

Shared state: extracted fields, lookups and validation results, read and written by several steps.



Seven invoices

	Invoice	What it presses on
S1	Clean, under threshold	Baseline. Compare model calls and tokens for the same outcome.
S2	Duplicate of a posted invoice	Is 'check before post' a guarantee or a request in a prompt?
S3	Unknown vendor	A failure with an obvious recovery path.
S4	9% over the PO, tolerance 2%	Where does a business rule live, and can you test it?
S5	Vendor service 503s twice	What does a retry cost, in calls and tokens?
S6	A credit note, not an invoice	The branch nobody drew.
S7	2,65,000 INR, above threshold	A human has to say yes, and is not online right now.



Run 1: the loop works

```
$ python run.py loop S2
== LOOP S2 -> escalated (duplicate)
model turn 1 in=491 out=44 Looking up the vendor and the PO.
tool lookup_vendor {"vendor_name": "Sri Balaji Packaging"} -> V-1002
tool lookup_po {"po_number": "PO-7782"} -> 42500.0
model turn 2 in=712 out=39 Checking for duplicates.
tool check_duplicate {... "invoice_number": "SBP-4471"} -> duplicate: true
model turn 3 in=903 out=52 Escalating: duplicate.
tool escalate_to_human {"reason": "duplicate"} -> EXC-0001
model calls=4 tokens in/out=2318/94 tool calls=4
```

Ask the room: what guaranteed the duplicate check ran before the post? Then scroll to rule 4 of the system prompt. **The loop got it right. The graph cannot get it wrong. Which one do you want on the payment run?**



Run 2: a flaky service

```
$ python run.py loop S5  
-> posted
```

```
model calls      6  
tokens in / out  3,339 / 136  
tool calls       6
```

```
turn 2: 'transient error, retrying'  
turn 3: 'transient error, retrying'
```

```
$ python run.py graph S5  
-> posted
```

```
model calls      1  
tokens in / out  156 / 60  
tool calls       6
```

```
vendor_lookup: RetryPolicy, attempt 3 ok  
no tokens spent on the decision
```

Same outcome. The loop paid five extra model calls to conclude 'try again', and would pay them again on every invoice during the outage. The retry policy is four lines and costs nothing per attempt. This is the direction in which a graph cuts tokens.

Figures are from the offline mock. Replace with the dry run before the session.



Run 3: waiting for a human

```
$ python run.py graph S7
== GRAPH S7 -> paused (waiting for human approval)
  note approval_gate INTERRUPTED: {"question": "Approve for payment?", "amount": 265000.0}
  note approval_gate resume later: python run.py resume S7-graph-41522 --approve --by <name>
$ exit                                     # process gone. go home.
```

```
$ python run.py resume S7-graph-41522 --approve --by Vinitha      # next morning, new process
note resume              resuming at ('approval_gate',) from checkpoint 1f1b20ba
tool post_to_ledger {"amount": 265000.0, "approved_by": "Vinitha"} -> LDG-00001
model calls=0 tokens in/out=0/0
```

The loop's answer to S7 is a ticket that says needs_approval. Ask the room how they would make a while loop wait until tomorrow. Every answer is a checkpointer, written by hand.



Run 4: the branch nobody drew

```
$ python run.py graph S6
model    extract      -> {"document_type": "credit_note", "amount": -5015.0, ...}
tool     lookup_vendor -> V-1002      tool  check_duplicate -> false
tool     post_to_ledger {"amount": -5015.0, "approved_by": "auto"} -> LDG-00001
== GRAPH S6 -> posted (a negative payable is now in the ledger)
```

```
$ python run.py loop S6
== LOOP S6 -> escalated (other)
'Document is a credit note against a
prior invoice, not a payable...'
model calls=2
```

```
$ python run.py hybrid S6
== HYBRID S6 -> escalated (other)
extract -> triage (bounded loop, 4 turns max,
no post_to_ledger in its tool list)
model calls=2
```

The model told you. Your graph had no edge to hear it. The fix is one edge and a loop inside a node with a turn budget and no access to the irreversible tool.

python run.py bench



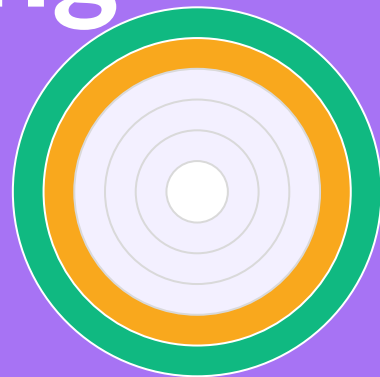
	Scenario	Loop	Graph	Hybrid
S1	Clean	posted (4 calls)	posted (1 call)	posted (1 call)
S2	Duplicate	escalated	escalated	escalated
S3	Unknown vendor	escalated	escalated	escalated
S4	PO mismatch	escalated	escalated	escalated
S5	Flaky service	posted (6 calls)	posted (1 call)	posted (1 call)
S6	Credit note	escalated	POSTED -5,015	escalated
S7	Needs approval	escalated (ticket)	paused, then posted	paused, then posted

Five rows agree. The two that split are the two signals: an irreversible action that needs a human, and a branch nobody drew.



PART D | FAILURE MODES, THE PRODUCTION STORY, THE SIGNALS

Decision: when a loop stops being enough



you are here



Two failure modes per shape

1

Loops that never terminate

A tool flakes, the model retries, the transcript grows, the turn budget is the only brake. You find out from the invoice, not from an alert.

2

Loops that lose the thread

Thirty turns in, context is long, and the model forgets it already posted. Nothing in the code remembers on its behalf.

3

Pipelines with an LLM bolted on

The shape was known all along and the model decides nothing. An agent call on an edge that never needed one.

4

The branch nobody drew

The extract node says `credit_note`. The router never looks at `document_type`. The graph does exactly what it was told, which is the problem.

From production: 1,500 layouts



Enterprise accounts payable, an ERP behind it

About 1,500 unique invoice layouts, 3 to 30 pages each. Targets: 95% field accuracy, 95% straight-through.

First instinct: one agent, one base prompt. It could not reach 95% across 1,500 layouts.

What got there was structure, not a better prompt.

Cluster layouts by visual similarity, about 10 clusters, one prompt per cluster

Extract from header and summary pages only: 60% less compute, same clustering accuracy

Five validation levels: schema, format, business rules, anomaly, cross-reference

Confidence-based routing: high goes through, uncertain goes to a human

The model extracts inside a node. The graph owns the routing.



The numbers behind it

1,500

unique invoice
layouts

95%

field accuracy and
straight-through
targets

~10

layout clusters, one
prompt each

60%

less compute from
header and
summary pages
only

5

validation levels
before a human
sees it

The lesson: the moment you can draw the clusters on a whiteboard, the model should stop deciding the route. Let it do the reading; you do the routing.



When to promote a loop

Three families of signals. Ours are the safety ones, because ours can move money.

General

The task sprawls and one context cannot hold it.

Step A must always lead to B, C or D by rule.

Specialists with a few tools beat one generalist with twenty.

Quality and cost

Work splits into distinct specialties.

Real independent fan-out, not a sequence drawn as boxes.

Different models or tools per step.

Control flow you can hand over as a diagram.

Safety (ours)

Parallel work that has to converge.

A human checkpoint before an irreversible action.

Recovery paths that differ by failure type.

Shared state several steps read and write.

Did you change paradigms?



CHECK YOURSELF

Are you engineering loops... or building a graph?

☐ 1 Are responsibilities split across specialized nodes?
No? You're still overloading a single loop.

☐ 2 Do nodes own clear state and context boundaries?
No? Context will leak and quality drops.

☐ 3 Are edges explicit, typed, and replacement-friendly?
No? Your system is brittle and hard to evolve.

☐ 4 Can parts run independently, scale, and fail safely?
No? You don't have a system—just a bigger loop.

HOW MANY DID YOU ANSWER "YES" TO?

0-1 YES

2-3 YES

4 YES

☒ STILL A LOOP
You're iterating, not decomposing.

☐ COMPOSING LOOPS INTO A GRAPH
You're on the right path—keep going.

☒ GENUINE PARADIGM SHIFT
You're building a graph distribution system.

★ DESIGN FOR COMPOSITION. BUILD FOR EVOLUTION. <

Before you draw anything, five questions

- 1 Can you draw the happy path without a branch you cannot name?
- 2 Does anything irreversible happen, and must someone approve it first?
- 3 Does anyone have to wait? Minutes, hours, a business day?
- 4 Do failures need different recoveries?
- 5 Do several steps read and write the same state?

Count the yeses. 0 to 1: write the loop. 2 or more: graph outside, loop where the judgment is.

Illustration: DevCompass

Usually, a hybrid



The orchestrator owns the topology. The loop governs how a node executes. Production systems are hybrids: bounded loops inside deterministic nodes. Knowing where to draw that boundary is most of the job.

Where to cut first: pull the verifier out into its own node with a fresh context. That one change fixes the most common failure before you commit to a topology.

The triage node from Run 4, as a pattern

Tum budget

for turn in range(1, 5). The graph escalates if it runs out.

Fewer tools

lookups and escalate only. post_to_ledger is not in the list.

Typed exit

the node returns state the graph can route on, not prose.

One job

work out what this document is. Nothing else.



What to take home

1

The repo

Loop, graph and hybrid on one task, seven scenarios, a bench, nineteen tests. Runs offline on a mock model, or on Claude with a key. Link in the chat.

2

The position

Start with the loop. Escalate on signals, not on a feeling. When you do, keep a bounded loop where the judgment lives, and pull the verifier out first.

3

Go deeper

One evening of the agents module in the Gen AI and Agentic AI Professional Program. Twelve weeks, live, hands-on, real systems.

If you have shipped an agent and it surprised you, **now is the time to argue with me.**

Questions in the chat or off mute. Ten minutes.

Thank You!

Navaneeth Malingan

www.nunnarilabs.com

Nunnari Academy is the training arm of Nunnari (நுண்ணறி) Labs
enquiry@nunnari.academy | 9042212130



ML.Cbe



 linktr.ee/nivu.ai

Connect with Me!



Thank you

enquiry@nunnari.academy | 9042212130

Nunnari Academy is the training arm of Nunnari (நுண்ணறி) Labs

Illustrations on slides 22, 23, 34, 42 and 57: DevCompass